

Impossibility results

Martín Soto

September 10, 2023

Recall that, when Ω 's behavior truly depends only on actions and not our algorithm, $\mathbf{A}$ can safely argmax away, and so no impossibility result exists. This deals with the other case, when Ω knows $\mathbf{A}$'s algorithm.

Non-average results

There are two free parameters: the decision tree, and the prior (or equivalently the Logical Inductor the agent uses). "Being more Updateful" can be hard-coded as part of the prior, so we can focus on just these two parameters.

For every prior (including Updateful ones) there is an arbitrarily bad adversarially chosen decision tree. The opposite doesn't hold: some decision trees aren't argmaxed by any prior (or even any program $\mathbf{A}$), nor argmaxed to within ε . There are trees where no updateless *finite* prior does as well as updateful ones. But there are also (infinite) trees argmaxed by an updateless finite prior.

So clearly we can't ask for impossibility results of the form "for all decision trees". We'll need to look at some notion of "average" amongst decision trees.

Average results

Say $\bar{U}_n(\mathbf{A})$ is the average reward $\mathbf{A}$ obtains across all decision trees of depth n , with at most n total positive reward per node, and logical coins of length at most n .

Definition 1 (Comparing averages). $\mathbf{A}$ does *better on average* than $\mathbf{A}'$ if $\bar{U}_n(\mathbf{A})$ eventually dominates $\bar{U}_n(\mathbf{A}')$ (as functions of n). Alternatively, we could impose stricter bounds on reward, and compute a finite $\lim_n \bar{U}_n(\mathbf{A})$ which we compare.

Theorem 1 (The sought-for impossibility result). Suppose $\mathbf{A}$ doesn't learn. Then, another $\mathbf{A}'$ that does learn does better on average.

Definition 2. We say $\mathbf{A}$ *doesn't learn* if, for any decision tree, when $\mathbf{A}$ faces (at two different times) two identical situations in different parts of that one tree, it takes the same actions.

Intuitively, if $\mathbf{A}$ acts "like this" in this problem (for example, a Counterlogical Mugging with this specific logical coin), it will act "like this" forever, no matter how many times the same situation is faced.

This definition of learning seems like a simpler predecessor to your "being interpretable as using an

improving prior". Indeed, we must first be able to notice something is changing at all.

But what do we mean by identical situations above? Here start the problems.

We still don't know how to deal with logical coins

What we mean is "isolated from the rest of the tree", that is, those subsequences of decision problems have no inter-dependencies (explicit cross-entanglements) with the rest of the tree.

But there's a problem: What if **A** thinks that a logical coin somewhere else talks about one of the two identical situations, but not the other? Even a completely Updateless **A** might act differently in those two situations.

This is a hard problem to solve (as usually happens with logical coins). A priori, we can't just say "assume **A** doesn't believe such correlations exist", because we're not assuming **A**'s internal structure in our Theorem statement. But actually, even if we were assuming it, and **A** was just our UDT, we wouldn't be able to say that either, because **A**'s beliefs about correlations are a mess! It might believe the correlation between this situation and that super complex logical coin is 0.02, and this will sometimes come into play and change the action it chooses. We don't know how to draw the boundary around evil logical coins.

In fact, the "boundary" we want to draw is clearly mind-dependent, so an objective boundary might not exist in general! It is conceivable that we could define a conservative objective class of non-evil logical coins so that, in the limit, all LIs eventually put arbitrarily low probability on those coins being correlated with our actions. But sadly, I think that's not the case. I think for any coin φ we can always construct an LI whose verdicts (and thus **A**'s actions) seem to us very correlated with φ (and thus, allegedly, that same LI will eventually notice this syntactic pattern).

We can instead try to change the mind, by assuming **A** is indeed our UDT, and then imposing (through infinite weight traders) that $\mathbb{Q}_n(\varphi|\mathbf{A}(\mathcal{Q}) = a) = \mathbb{Q}_n(\varphi|\mathbf{A}(\mathcal{Q}) = a')$, for all logical coins φ that appear in the tree. But this, of course, will make our agent miss out on many intuitively exploitable correlations. We'd like a way to separate cases in which we want $\mathbb{Q}_n$ to entertain the correlation, from those in which we don't. But it becomes clear again that the metric we actually care about to separate those is performance: "in this concrete case, does entertaining it lead to higher reward?" And we, from the outside, can objectively compute that: for any σ a subset of the logical coins of the tree, just run the $\mathbb{Q}_n$ that entertains exactly those correlations, and check how well it does, and then implement the best one. But the agent, from within its prior, will never "reason through changing its σ ": if it believes a correlation to exist, erasing it so that it acts accordingly will seem bad in expected value (except when there are non-cartesian reasons to change it, such as Ω explicitly punishing that belief). The alternative is "just putting a mechanism on top of it which forces it to change", and that's basically a less general version of $\text{LIA}(\sigma)$, that is, Policy Selection.

So what if we just consider trees without logical coins? Well, we still got entanglements, which from the agent's perspective are nothing more than big logical coins they might be correlated to, so the same problem arises.

Back to the empirical case

So what if we only consider empirical coins? Then again **A** can just argmax the tree, or at least argmax to within ε maximum reward, since outcome only depends on its actions (as when Ω 's LI only looks at actions).

Is it nonetheless true that, on average, the argmax strategy *learns* as defined above? No: if two situations are truly identical, the argmaxed strategy will be the same for both.

But that's just because we've assumed away the central problem of empirical updatelessness: choosing the prior (since in our framework the agent has complete access to the decision tree). If we reintroduce this (so that the agent has a probability distribution over which tree it's in), we might get the desired result.

In fact, it's almost immediate, because in our way to assess reward (take the limit) we give the same weight to all trees (we just average their rewards), while an agent with any prior will give arbitrarily low weight to an infinite set of trees. Of course, if we average differently, for example with a simplicity prior, then indeed the agent with that prior will perform best. It seems like we're just saying something tautological here: We're baking in "the right prior" when defining "average", and then punish agents with a different prior (and if we take the average uniformly as above, it won't correspond to any simplicity prior, so we punish all agents).

This seems like a fundamental problem of Expected Value Maximization (that it shoots for the big wins and thus most times gets stuck in bad situations, unlike Kelly Betting). After all, our agent is just performing straightforward EVM across possible decision trees, and also across possible empirical coin tosses within those trees, at the same time.

What if instead, like Kelly Bettors, our agent maximized the logarithm of reward? Well, then behavior will change depending on where we set the reward zero point (the starting wealth). When the agent starts with low wealth (relative to that in the tree), it will indeed act more risk-averse. In the limit of high starting wealth, it will just act as an EVM.

For example, when facing a sequence of Counterfactual Muggings (say, on a fair coin), the agent will either not get mugged in any one, or get mugged in some, or get mugged in all of them, depending on its starting wealth.¹

Bottlenecks

As is evident above, the bottleneck to proving interesting things is having better models of logical coins, that is logical correlation, that is logical uncertainty. This has also proved a bottleneck for building agents. This informs my intuitions that we need radically new ideas (for example a more realist picture of logical uncertainty) to obtain some things we wanted.

¹This can immediately be applied to the Logical case. But I'd like a more native way to express "doubting your imperfect opinions on logic (and as a consequence re-deriving some things, or propagating your uncertainty across your web of logical beliefs)", which is not just "taking all of that probability distribution as gospel, and then adding things on top to make it artificially more hedge-y". See **Bottlenecks**.

Humbler results

For any LI $\mathbb{Q}$, and n , consider these two algorithms. $\mathbf{A}$ (when facing a decision tree of depth n) just runs $\mathbb{Q}$ for n steps, and then freezes that prior and uses it forever. $\mathbf{A}_f$ does the same, but hard-codes into the prior "after $f(n)$ steps, become updateful". Notice $\mathbf{A}$ and $\mathbf{A}_{n+1}$ always take the same actions (but they might not perform exactly equivalently, since their code is different).

Theorem 2 (A humbler sought-for impossibility result). There is f such that $\mathbf{A}_f$ does better than $\mathbf{A}$ on average.

I suspect $n/2$ already works (and many other natural functions), since in only a few trees does $\mathbf{A}$ get high reward (for paying up in Counterfactual Mugging or whatever), while on "the average random tree" this doesn't happen, and it's better to be updateful to adapt to this randomness (and randomness grows with n). But I don't have a proof yet.

I do worry that, as above, we are just "baking in" the notion of the correct prior over trees into our definition of average (uniform averaging). We no longer have a fixed prior, but we do have a fixed $\mathbb{Q}$, that will have certain finite-time biases in building priors (even if the priors also depend on the decision tree that $\mathbb{Q}$ is seeing).

We can also consider $\mathbf{R}_f$ (for Refreeze) which "after $f(n)$ steps, refreezes the prior", and other generalizations.